# Algorithms

## Lecture #40

Syed Hamed Raza

# Difkstra’s Algorithm

## Dijkstra’s Algorithm

# Dijkstra's Algorithm - Correctness

## *Dijkstra's Algorithm - Correctness*

Proof of Minimality by Induction

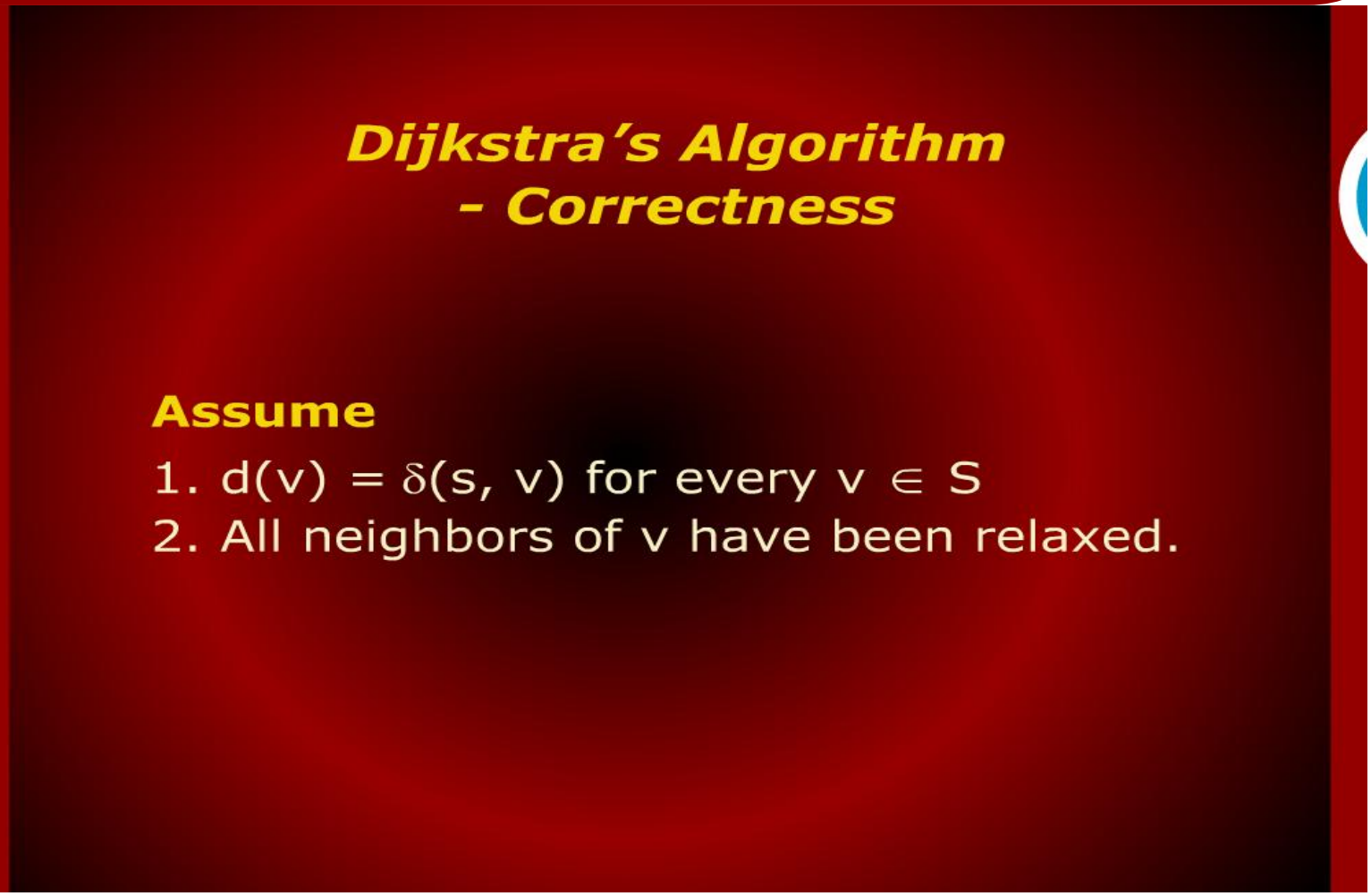
Definitions

$\delta(s, v)$: the minimal distance from s to v.

Base case

1. $S = \{s\}$
2. $d(s) = 0$, which is $\delta(s, s)$

# Dijkstra's Algorithm - Correctness



# Dijkstra's Algorithm - Correctness

## *Dijkstra's Algorithm - Correctness*

**Proof**

- If $d(u) \leq d(u')$ for every $u' \in V$ then $d(u) = \delta(s, u)$, and we can transfer u from V to S, after which $d(v) = \delta(s, v)$ for every $v \in S$.
- We do this as a proof by contradiction.

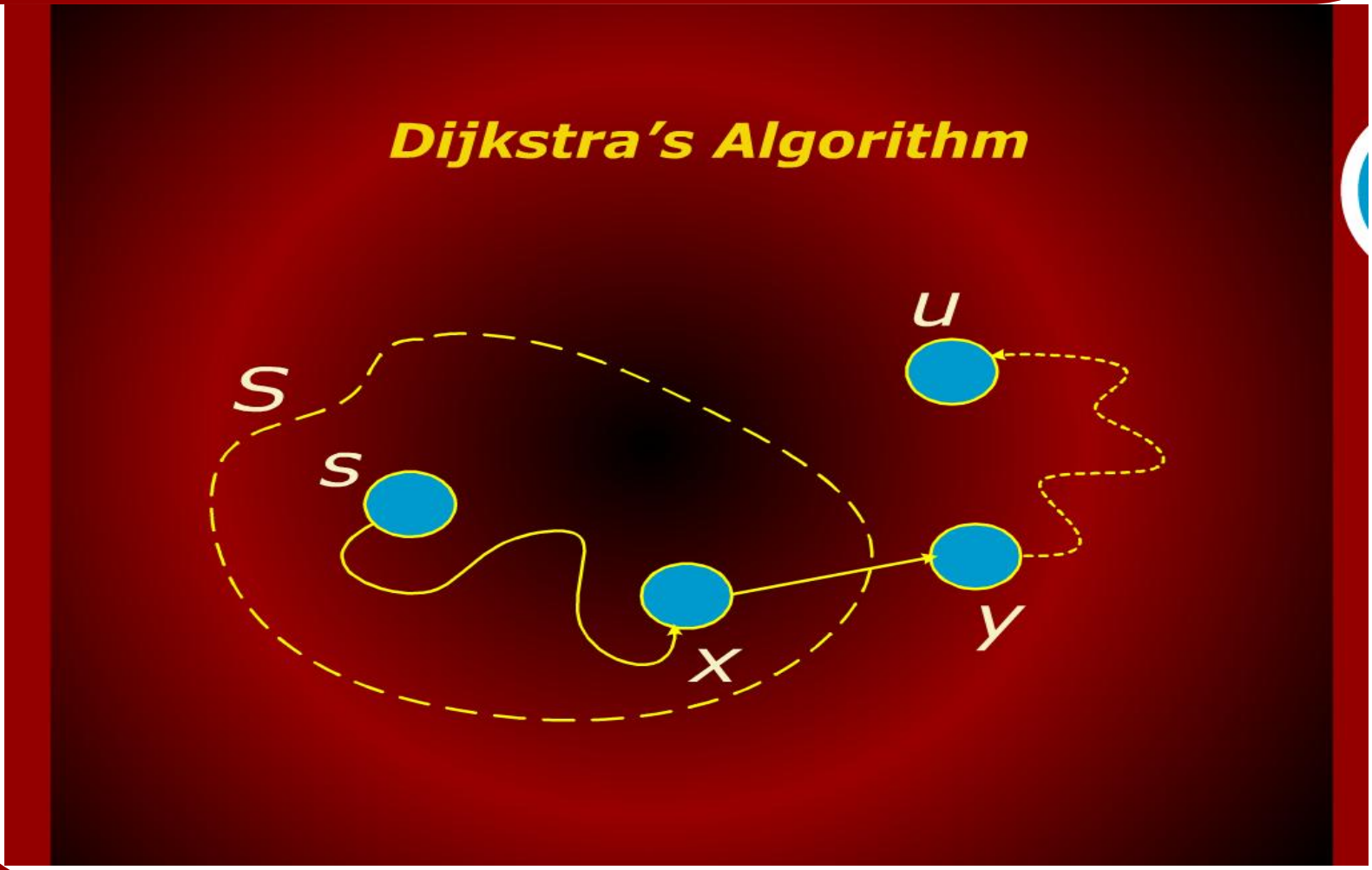
# Dijkstra's Algorithm - Correctness

## *Dijkstra's Algorithm - Correctness*

**Suppose that $d(u) > \delta(s, u)$.**

- The shortest path from s to u, p(s, u), must pass through one or more vertices exterior to S.
- Let x be the last vertex inside S and y be the first vertex outside S on this path to u.
- Then
  $p(s, u) = p(s, x) \cup \{(x, y)\} \cup p(y, u)$.

# Dijkstra's Algorith



# Dijkstra's Algorithm - Correctness

## *Dijkstra's Algorithm- Correctness*

- The length of p(s, u) is $\delta(s, u) = \delta(s, y) + \delta(y, u)$.
- Because y was relaxed when x was put into S, $d(y) = \delta(s, y)$ by the convergence property.
- Thus $d(y) \leq \delta(s, u) \leq d(u)$.
- But, because d(u) is the smallest among vertices not in S, $d(u) \leq d(y)$ .
- The only possibility is $d(u) = d(y)$, which requires $d(u) = \delta(s, u)$ contradicting the assumption.

# Dijkstra's Algorithm - Correctness

## *Dijkstra's Algorithm - Correctness*

- By the upper bound property, $d(u) \geq \delta(s, u)$.
- Since $d(u) > \delta(s, u)$ is false, $d(u) = \delta(s, u)$, which is what we wanted to prove.

**What has been proved**

- If we follow the algorithm's procedure, transferring from V to S, the vertex in V with the smallest value of d(u) then all vertices in S have $d(v) = \delta(s, v)$

# Bellman-Ford Algorithm

## *Bellman-Ford Algorithm*

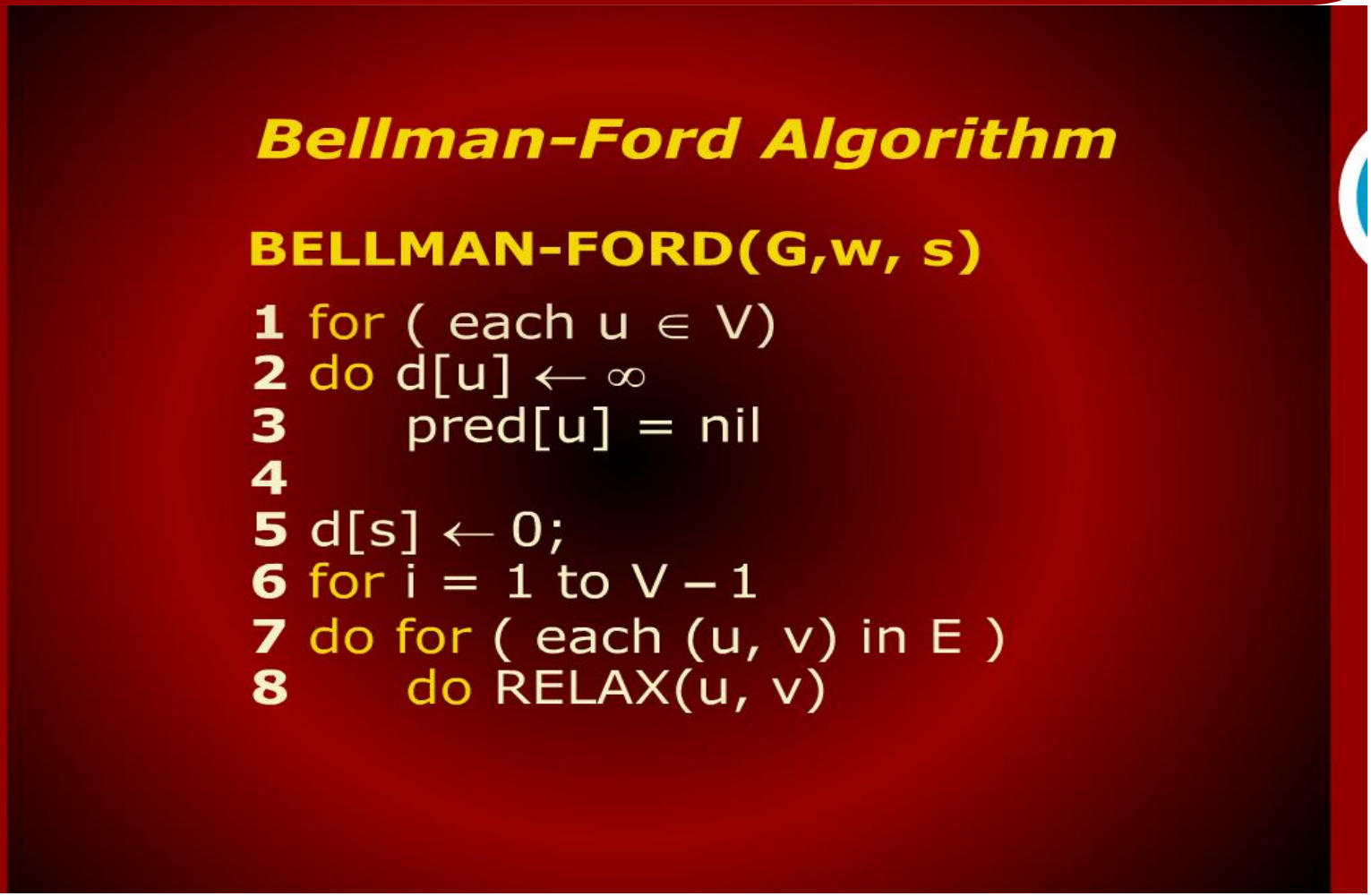
- Dijkstra's single-source shortest path algorithm works if all edges weights are non-negative and there are no negative cost cycles.
- Bellman-Ford allows negative weights edges and no negative cost cycles.
- The algorithm is slower than Dijkstra's, running in $\Theta(VE)$ time.

# Bellman-Ford Algorithm

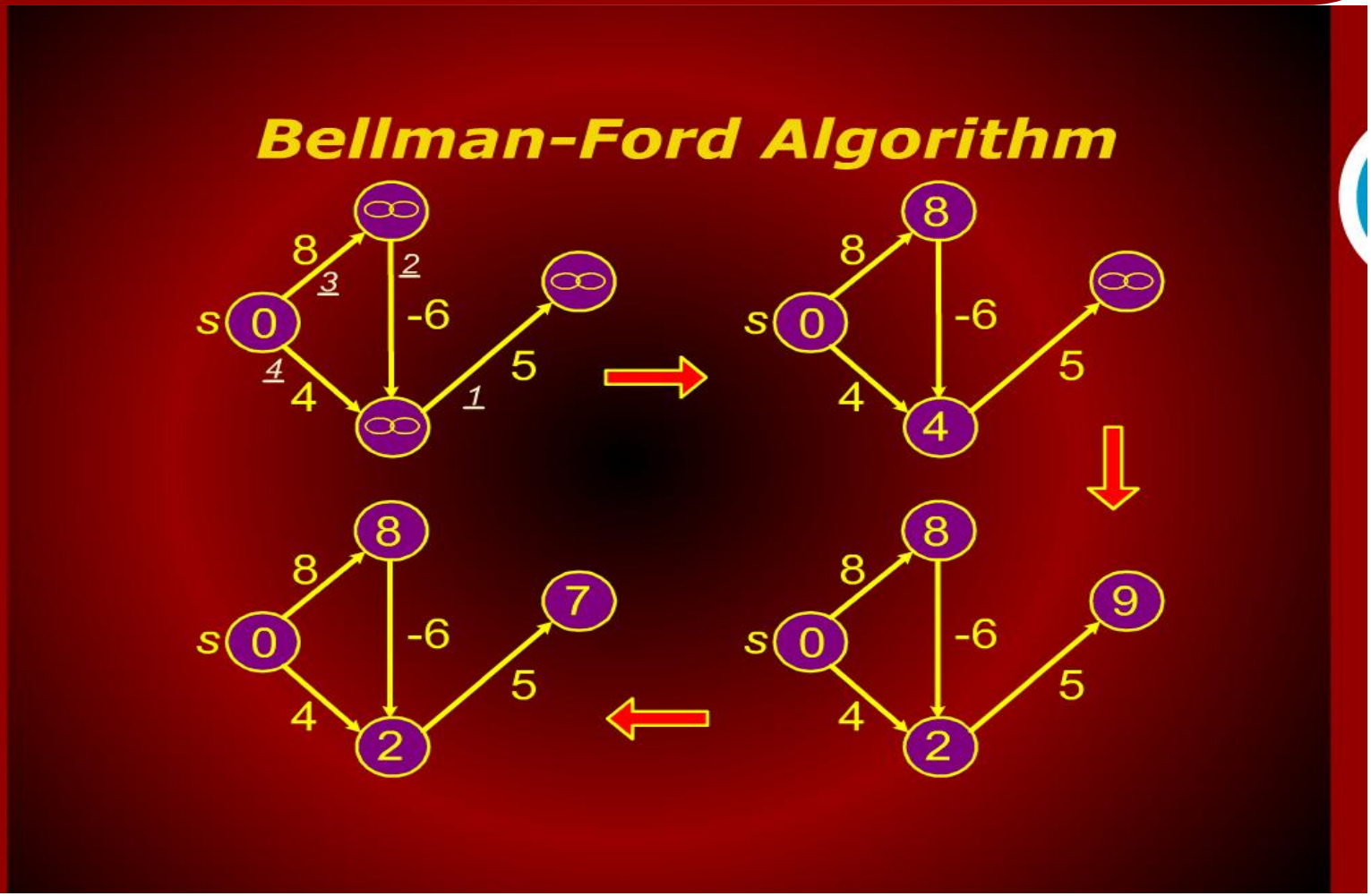
## *Bellman-Ford Algorithm*

- Like Dijkstra's algorithm, Bellman-Ford is based on performing repeated relaxations.
- Bellman-Ford applies relaxation to every edge of the graph and repeats this V – 1 times.

# Bellman-Ford Algorithm



# Bellman-Ford Algorithm



# Bellman-Ford Algorithm